

AI Implementation - Configuration Module

Functional Context

Solution proposal

High level summary of the proposal

Sequence diagram

Configuration

API changes

Datamodel

Overview Diagram



Functional Context

The Configuration Chatbot Assistant is an AI-powered feature in the Configuration module that helps implementation teams understand and configure business rules and parameters.

Problem Statement:

Implementation teams need to configure complex business rules (accrual, billing, campaigns, tier transitions, etc.) and parameters (currencies, expiration dates, partner activities, etc.). The documentation is in Confluence and is difficult to search and navigate, slowing setup and increasing errors.

Solution:

The chatbot provides a conversational interface within the Configuration UI. It uses **Retrieval-Augmented Generation (RAG)** with **Azure OpenAI** to search a knowledge base built from Confluence documentation. Users can ask questions in natural language and get contextual answers.

User Experience:

The chatbot is accessible via a modal or panel in the Configuration module. Users can ask questions like "How do I configure tier benefits?" or "What are the criteria for ACCR_PTS_EXPIRY_DATE?". The assistant searches the knowledge base and returns formatted answers, optionally linking to specific parameters for quick navigation.

Solution proposal

High level summary of the proposal

Implementation:

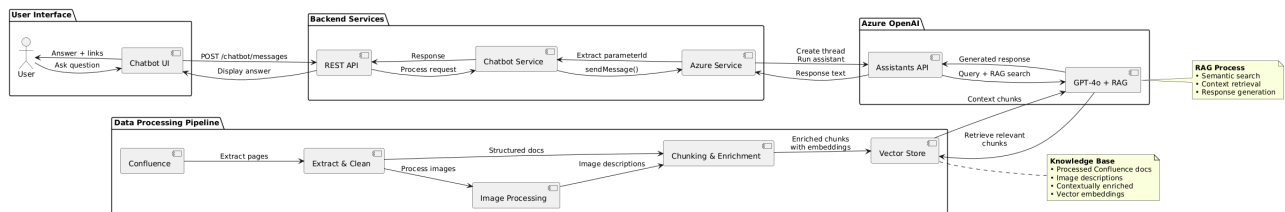
- Stateless design: each request creates a new thread
- Document pipeline: Confluence extraction → HTML cleaning → image-to-text conversion → contextual chunking → vectorization → Azure vector store
- Conversation history: passed in request payload, maintained client-side

Technology Stack:

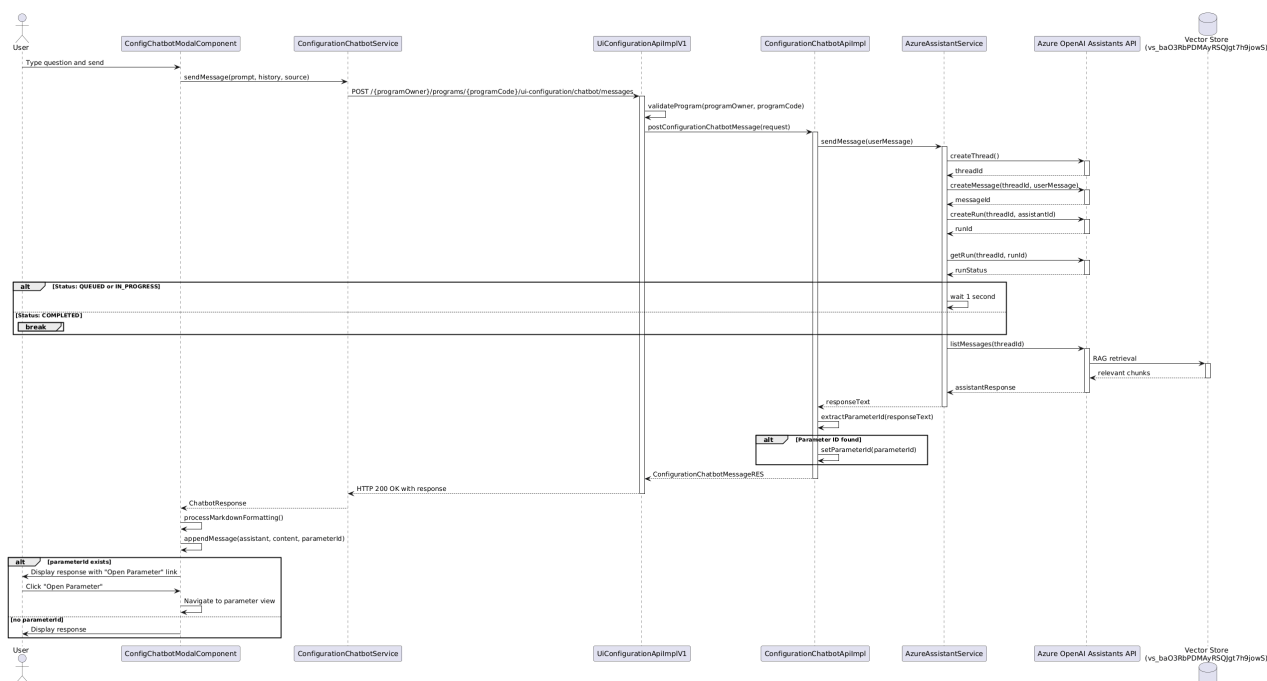
- Frontend: Angular, RxJS, Markdown rendering
- Backend: Java (JAX-RS), Azure OpenAI Assistants SDK 1.0.0-beta.3
- Knowledge Base: Azure OpenAI Vector Store with GPT-4o model
- Configuration: Environment variables (`AZURE_OPENAI_ENDPOINT` , `AZURE_OPENAI_ASSISTANT_ID`)

Data Flow:

User query → Frontend service → REST API → `AzureAssistantService` → Azure Assistants API (thread creation, message, run) → RAG retrieval from vector store → Response → Frontend display.



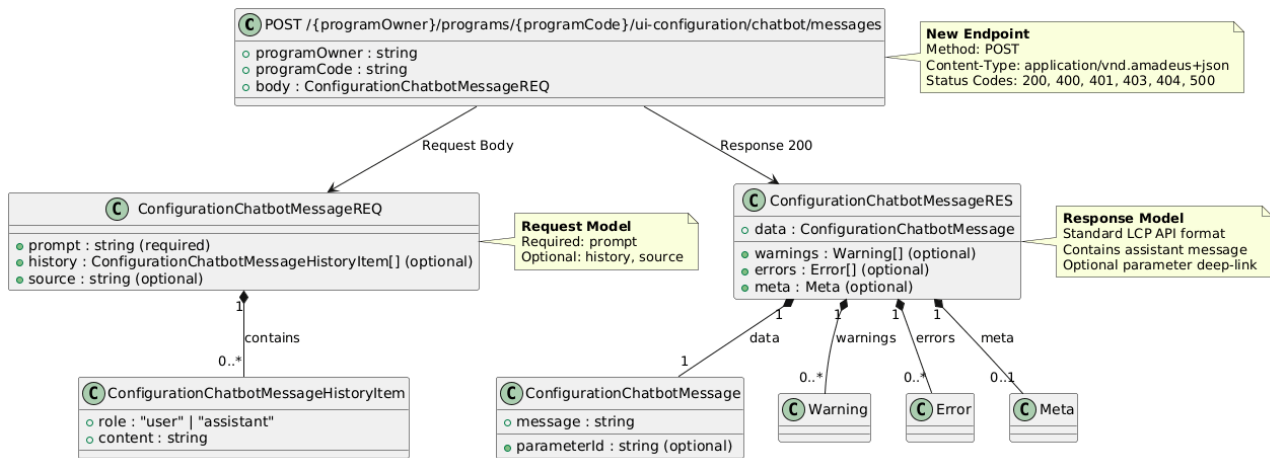
Sequence diagram



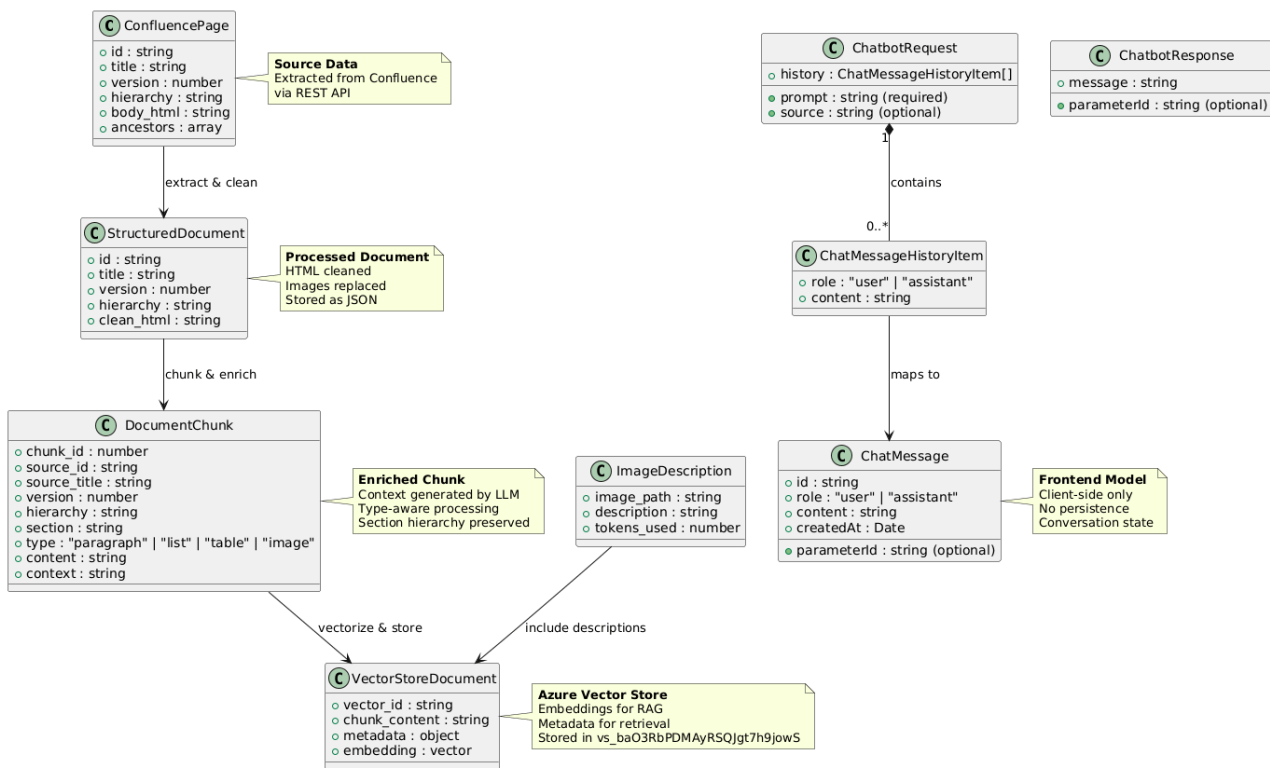
Configuration

- Frontend: Angular chatbot components (`ConfigChatbotModalComponent` , `ConfigChatbotPanelComponent`) in the Configuration module
- Backend: REST endpoint `POST /{programOwner}/programs/{programCode}/ui-configuration/chatbot/messages` implemented in `ConfigurationChatbotApiImpl`
- AI Service: Azure OpenAI Assistants API with RAG using a pre-configured assistant (`asst_69FwM9HW4I5t395wTu9N2a79`) and vector store (`vs_ba03RbPDMAyRSQJgt7h9jowS`)
- Authentication (*NEED TO BE IMPROVED*): Azure Access Token that expires in 1 hour. Cant do better because of lake of accesses

API changes



Datamodel



Overview Diagram

